



Universidade Federal do
Recôncavo da Bahia

FIFO ARCHITECTURES FOR DIGITAL SYSTEMS

Dr. Igor D S Miranda
Federal University of
Recôncavo da Bahia, Brazil



PRESENTATION OUTLINE

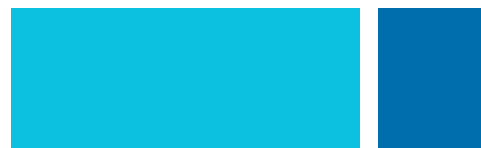


01 FIFO BASICS

02 APPLICATIONS

03 SYNCHRONOUS FIFO DESIGN

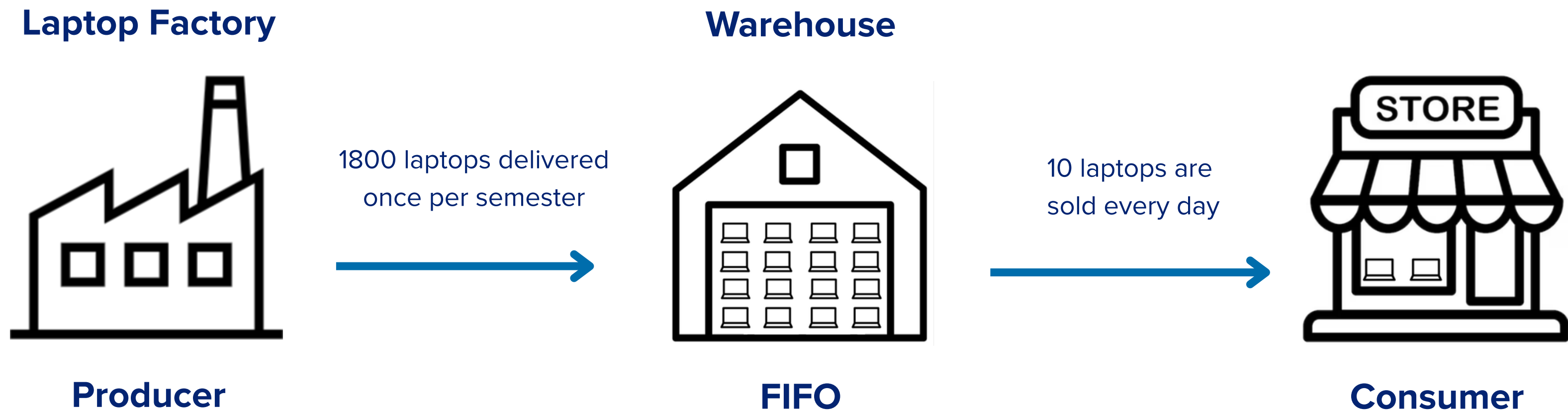
04 ASYNCHRONOUS FIFO DESIGN



FIFO BASICS

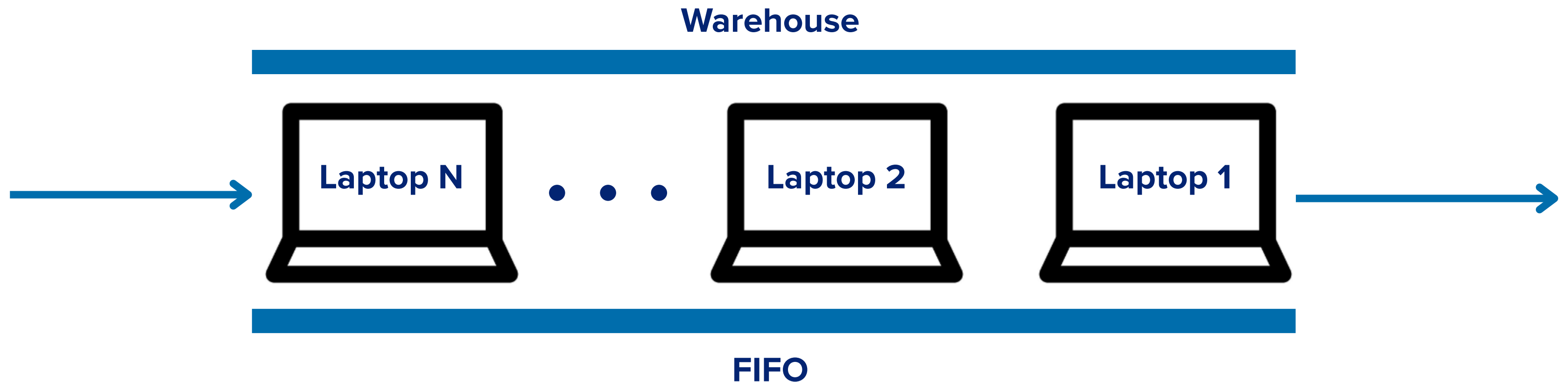
WHAT IS A FIFO?

A First-In, First-Out (FIFO) buffer that temporarily stores data in the order it was received.

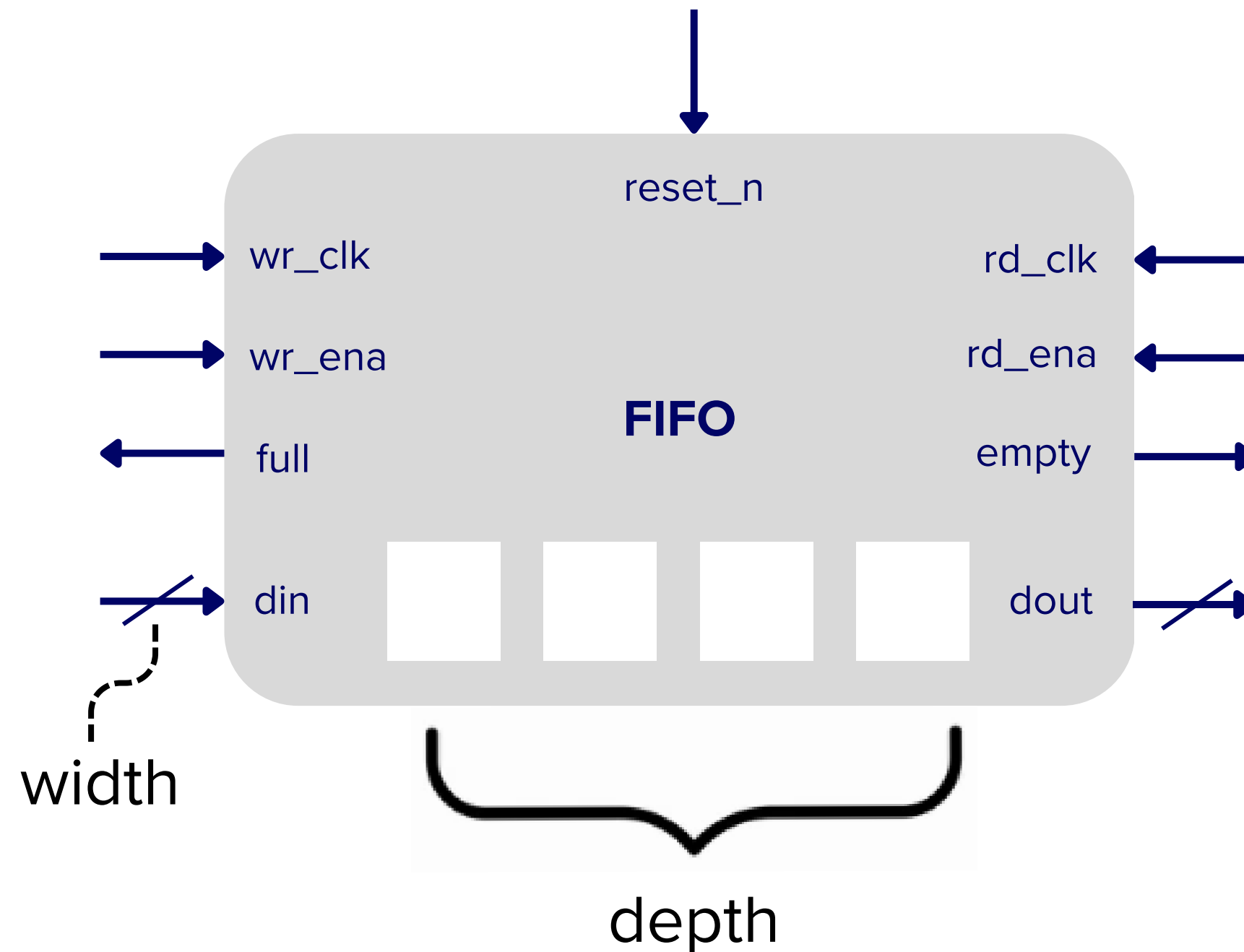


WHAT IS A FIFO?

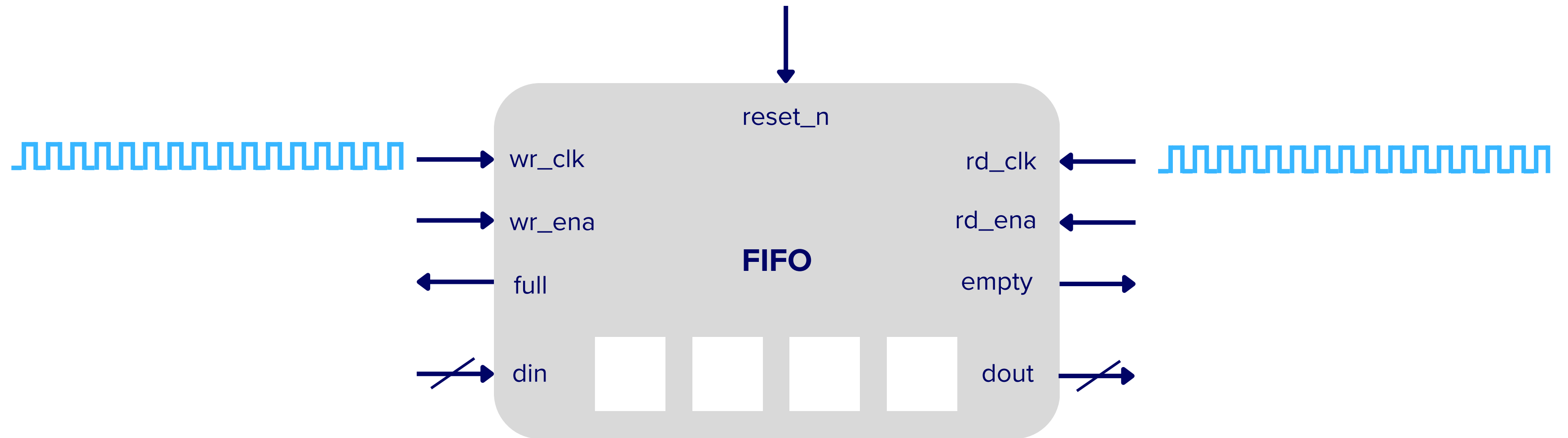
A First-In, First-Out (FIFO) buffer that temporarily stores data in the order it was received.



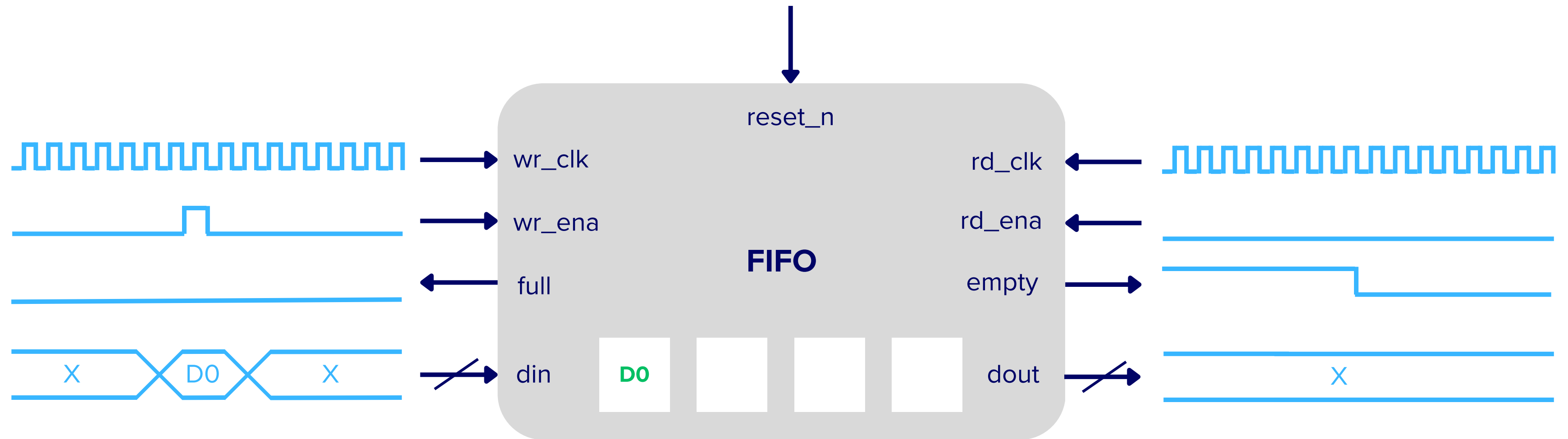
FIFO MODULE OVERVIEW



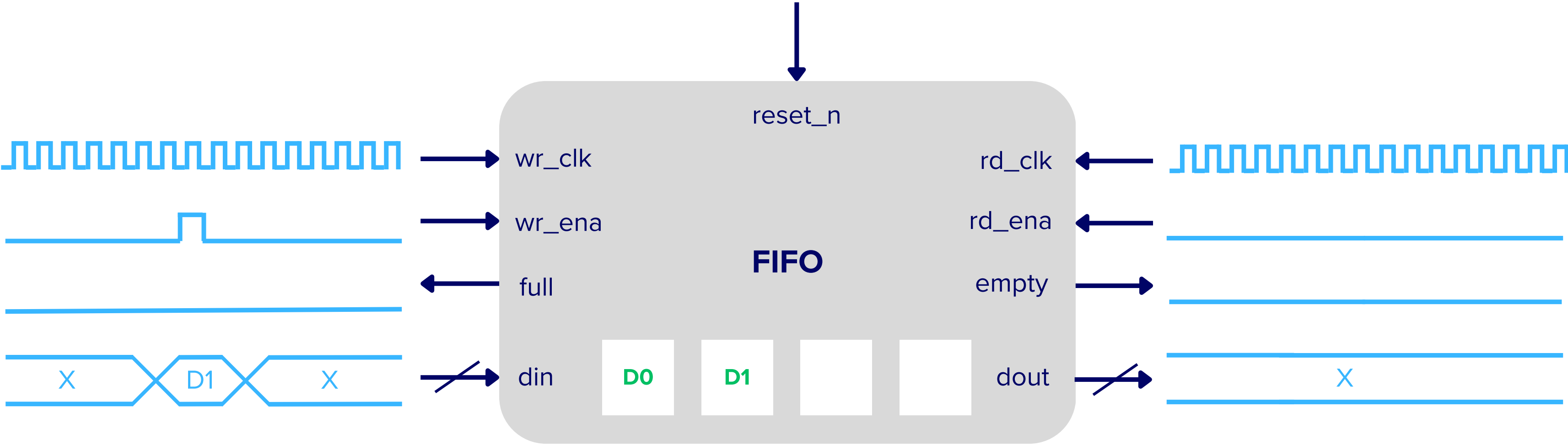
FIFO BASIC BEHAVIOR



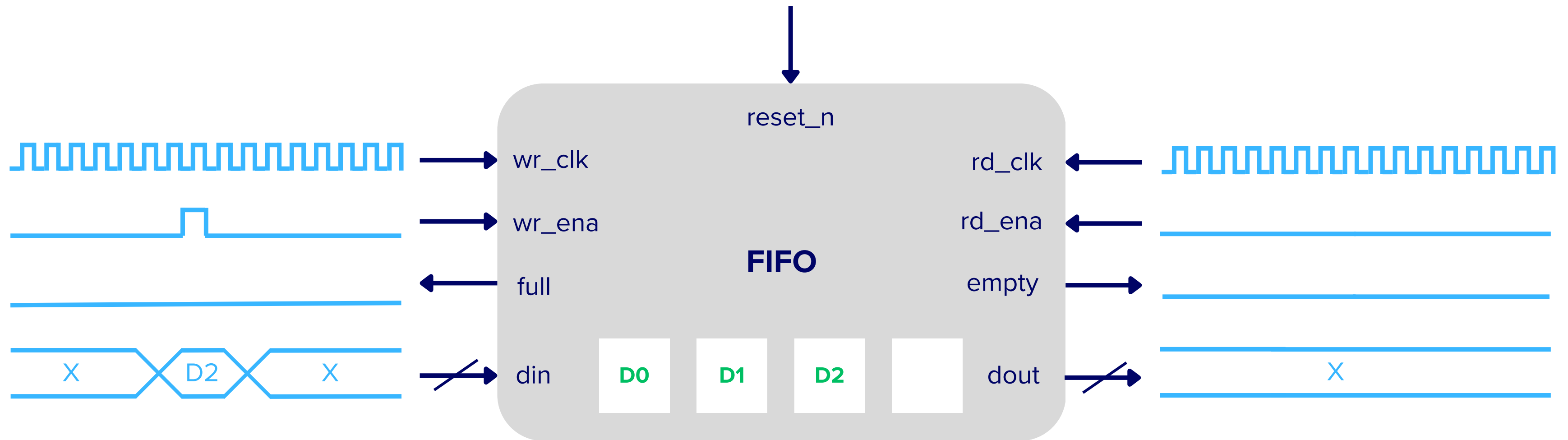
FIFO BASIC BEHAVIOR



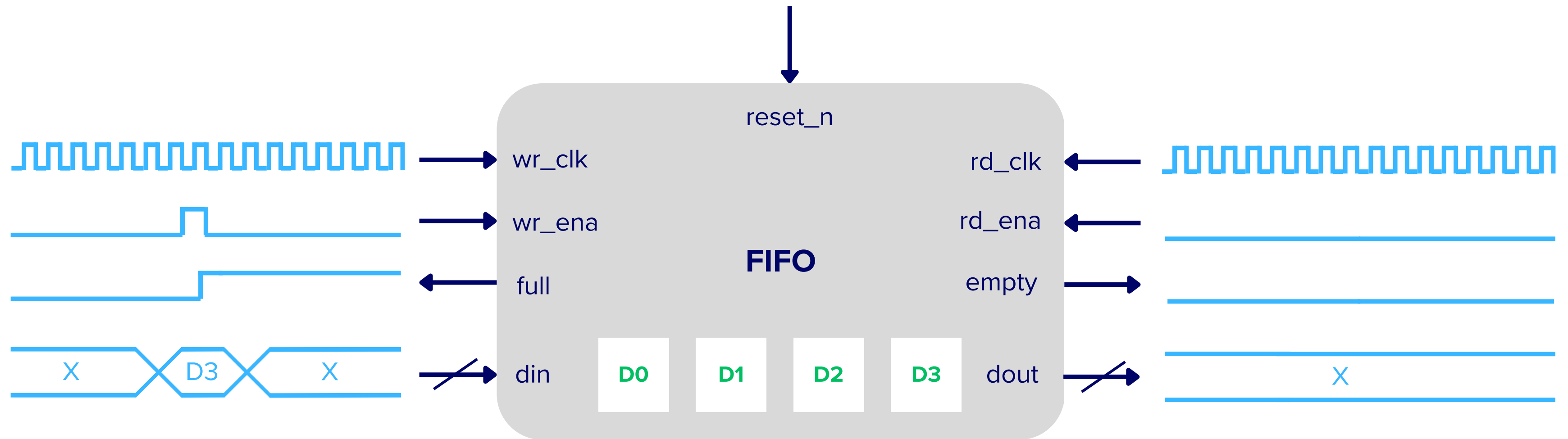
FIFO BASIC BEHAVIOR



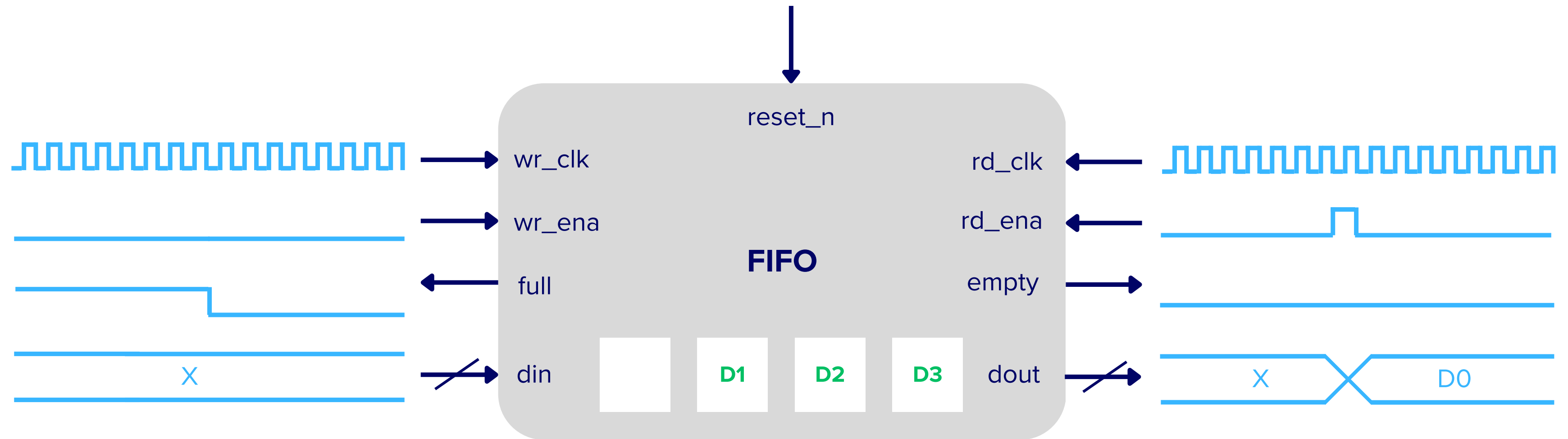
FIFO BASIC BEHAVIOR



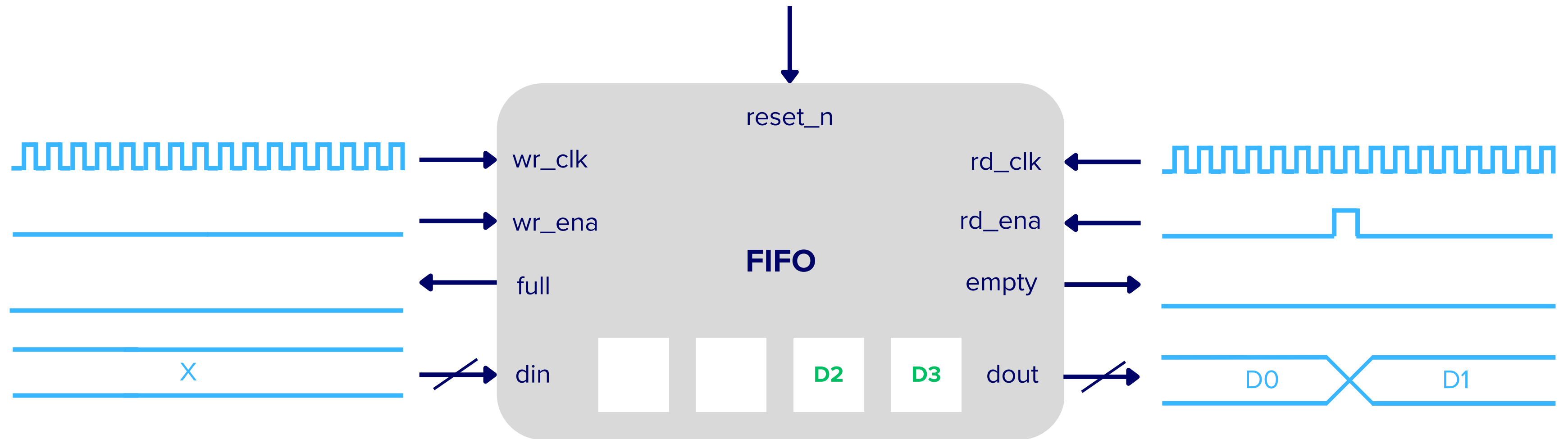
FIFO BASIC BEHAVIOR



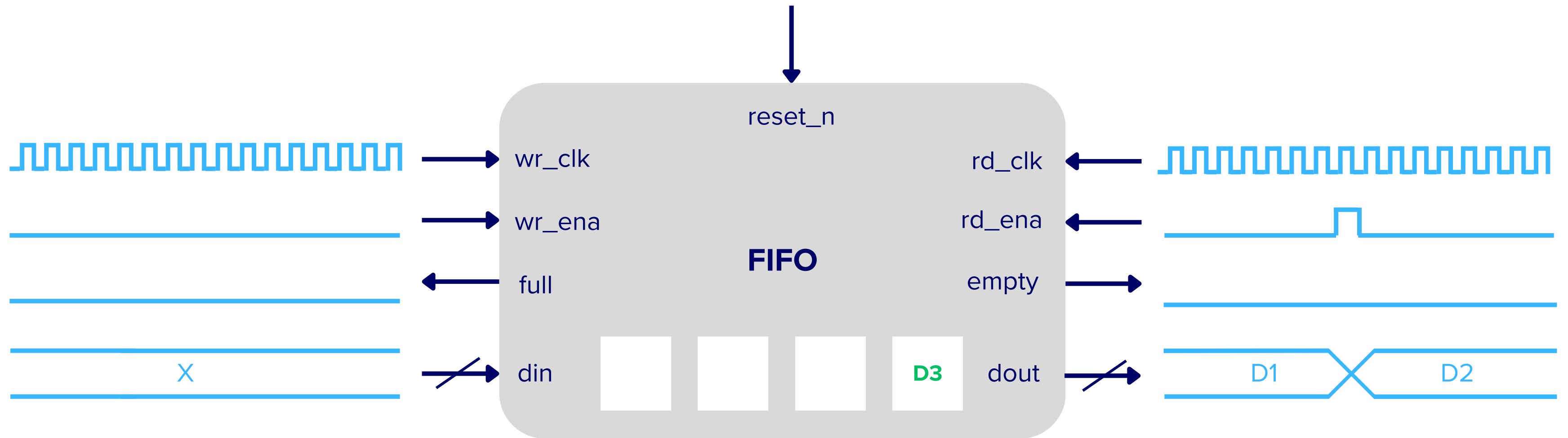
FIFO BASIC BEHAVIOR



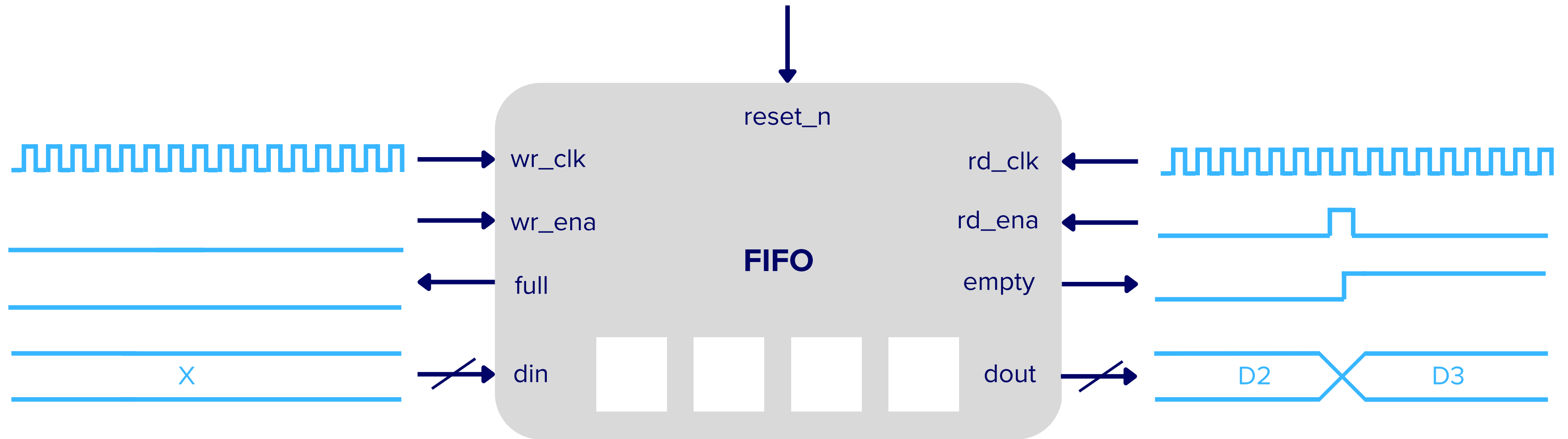
FIFO BASIC BEHAVIOR



FIFO BASIC BEHAVIOR



FIFO BASIC BEHAVIOR



FIFO APPLICATIONS

USE CASE 1: TEMPORARY DATA STORAGE BETWEEN PRODUCER AND CONSUMER



- Buffers data between independent blocks, whether they operate at fixed or irregular rates.
- *Example: A processor writes log messages to a FIFO at irregular intervals; a UART transmits them steadily.*



USE CASE 2: CLOCK DOMAIN CROSSING

- Enables safe communication across asynchronous clock domains.
- *Example: An FPGA logic block at 100 MHz sends data to a peripheral running at 25 MHz.*

USE CASE 3: FRAMED PROCESSING

- Some systems consume or produce data frames, not individual elements.
- *Example: A network processor reads entire Ethernet frames from a FIFO, waiting for End-of-Packet (EOP) marker before processing.*

SYNCHRONOUS

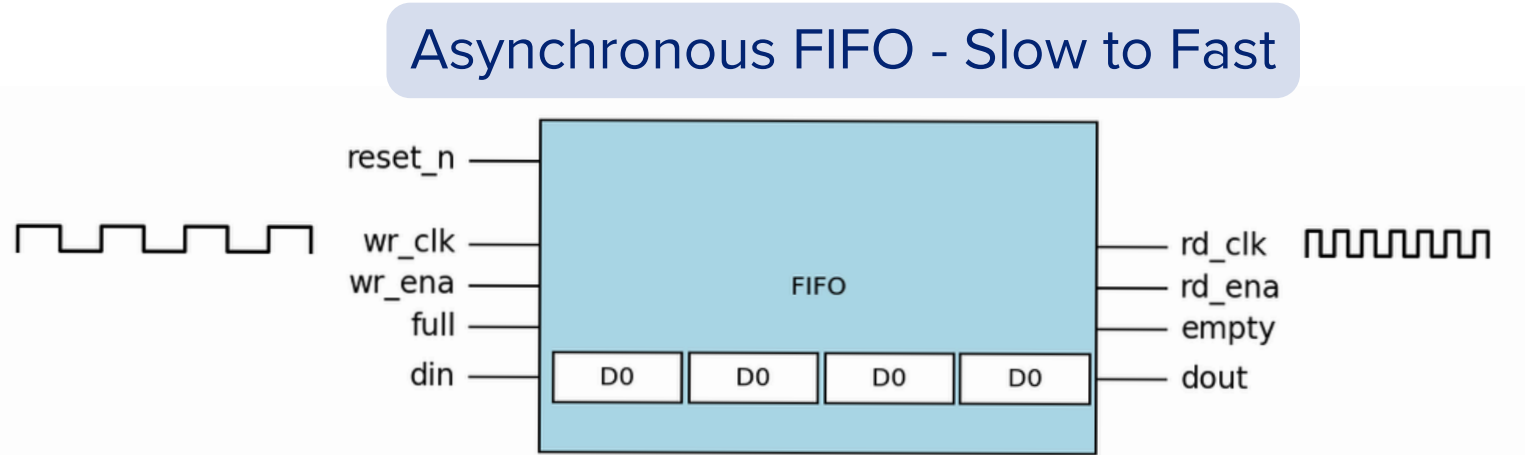
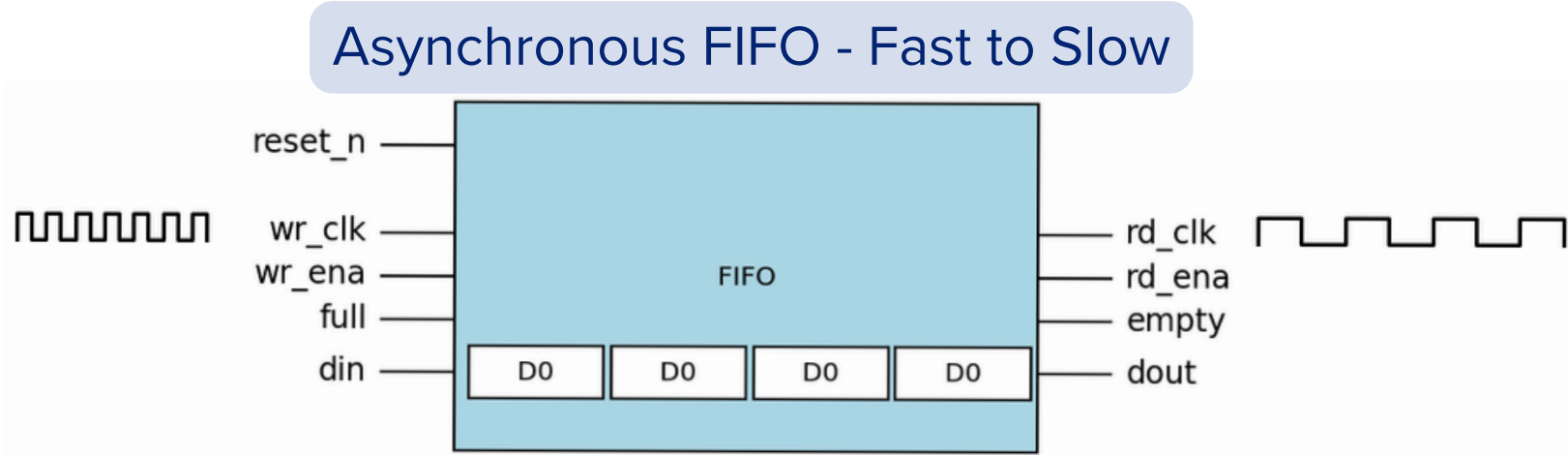
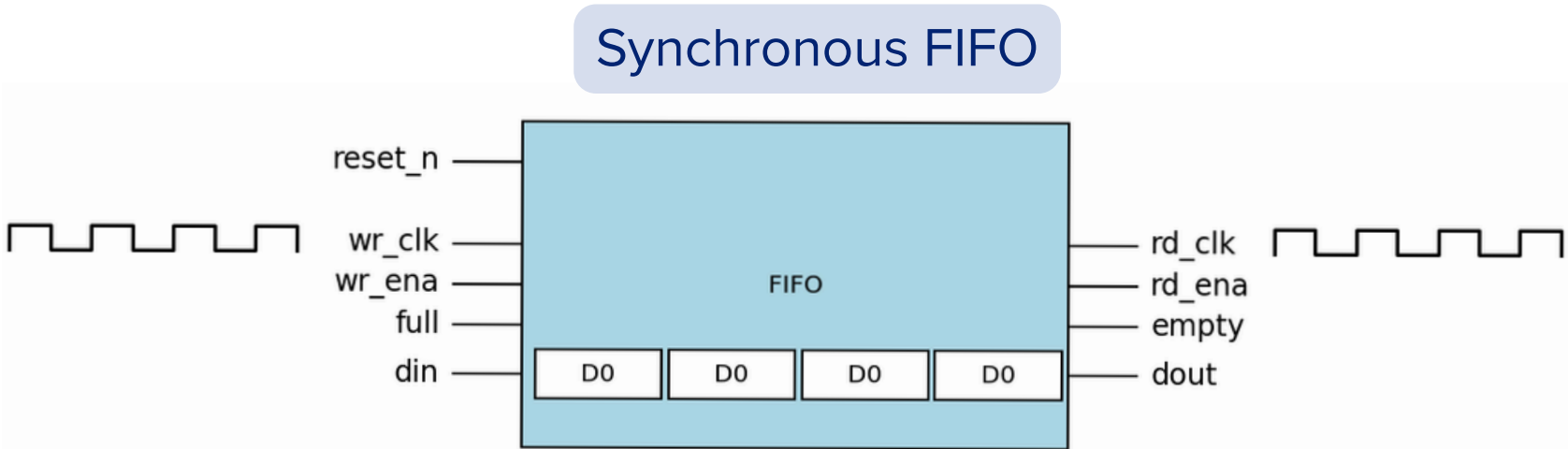
AND

ASYNCHRONOUS

FIFO DESIGNS

SYNCHRONOUS VS ASYNCHRONOUS FIFO DESIGNS

Feature	Synchronous	Asynchronous
Clock Domains	Single clock for both read and write	Separate clocks for read and write
Complexity	Lower — simpler control logic	Higher — needs careful metastability mitigation
Latency	Lower, due to synchronous timing	Slightly higher due to synchronizer delays

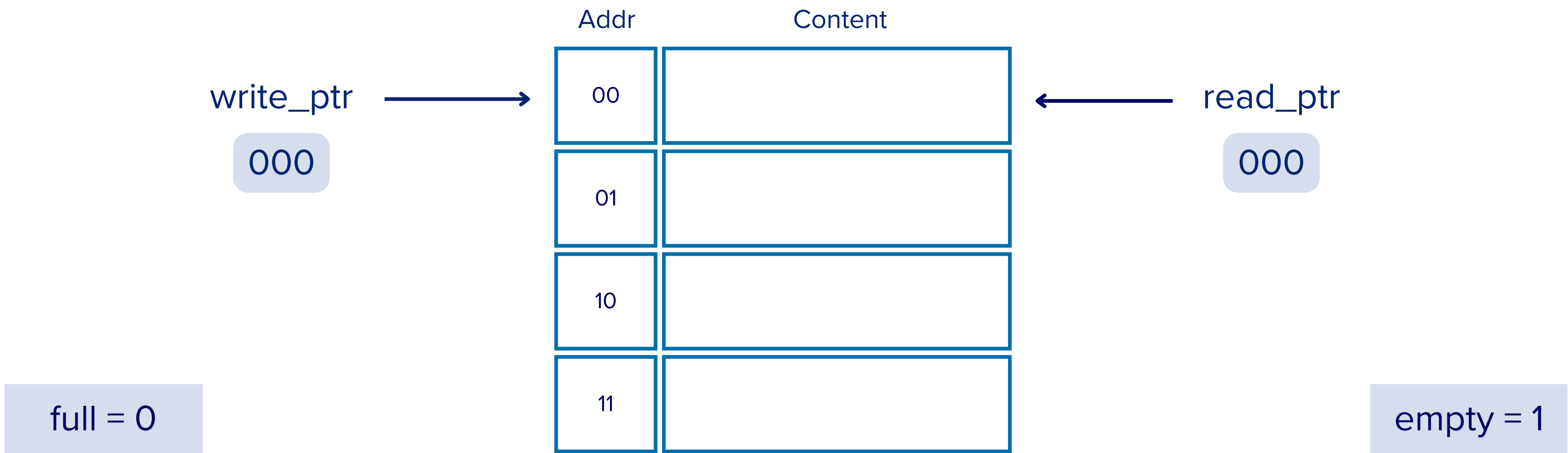


SYNCHRONOUS FIFO DESIGN

* Some images and source-codes in this section were adapted from Cadence training course

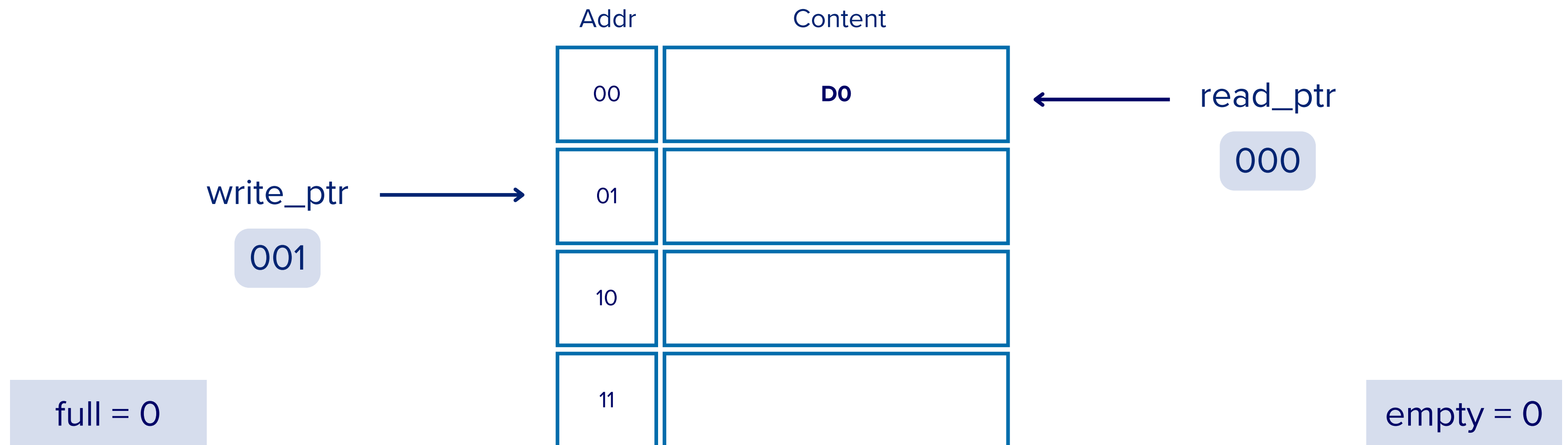
SYNCHRONOUS FIFO DESIGN

At its core, a FIFO uses a circular buffer with **dual-port memory**, **write_ptr**, and **read_ptr**.



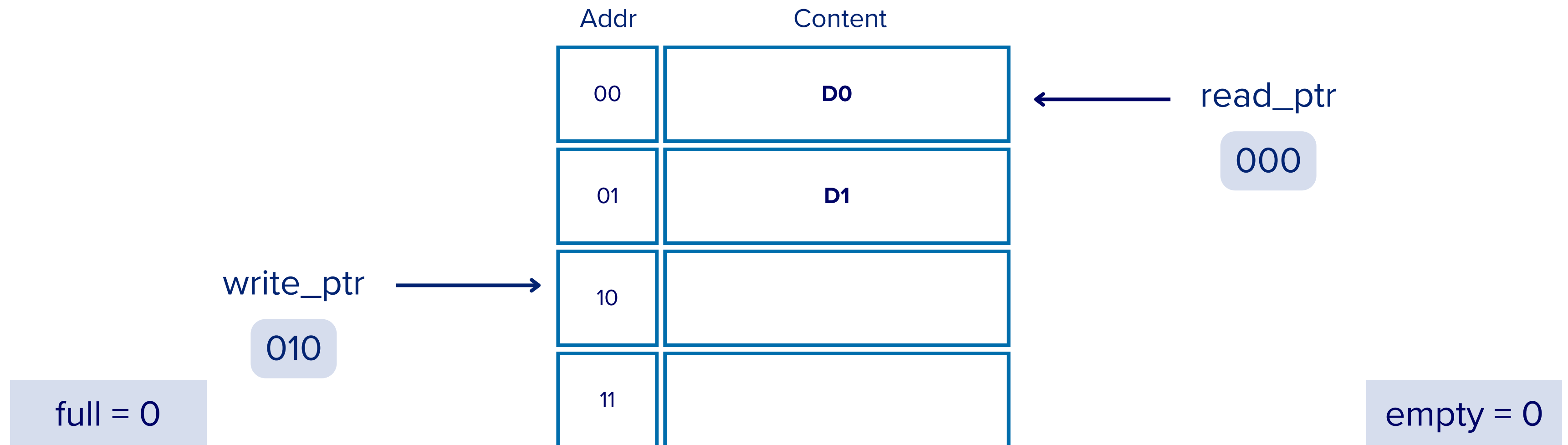
SYNCHRONOUS FIFO DESIGN

- 1st write
- empty flag goes low



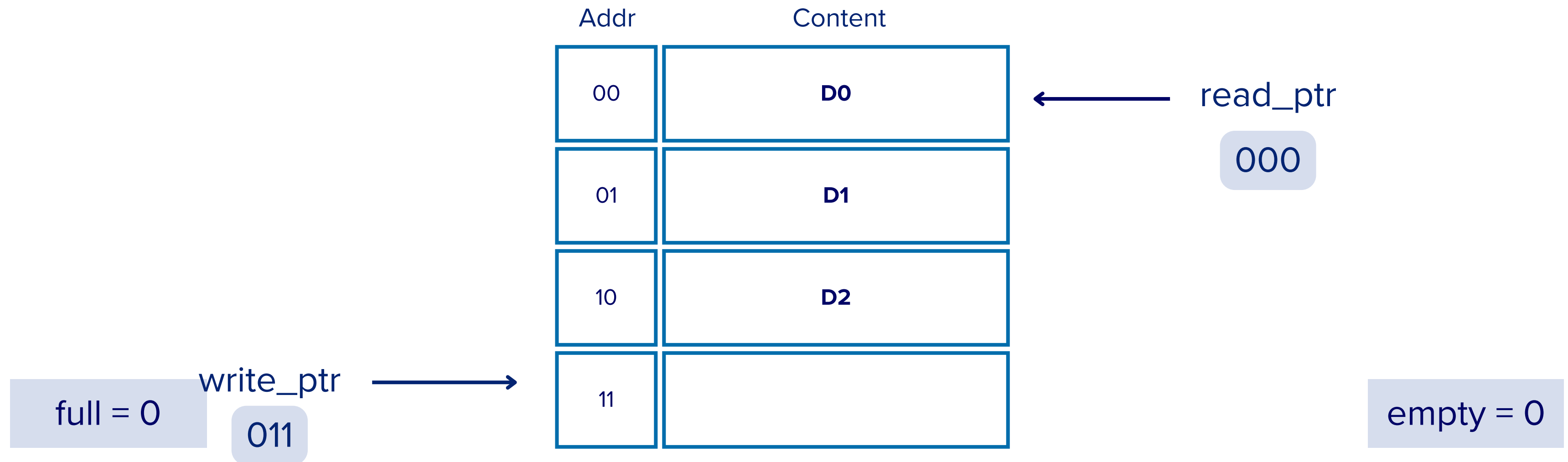
SYNCHRONOUS FIFO DESIGN

- 2nd write



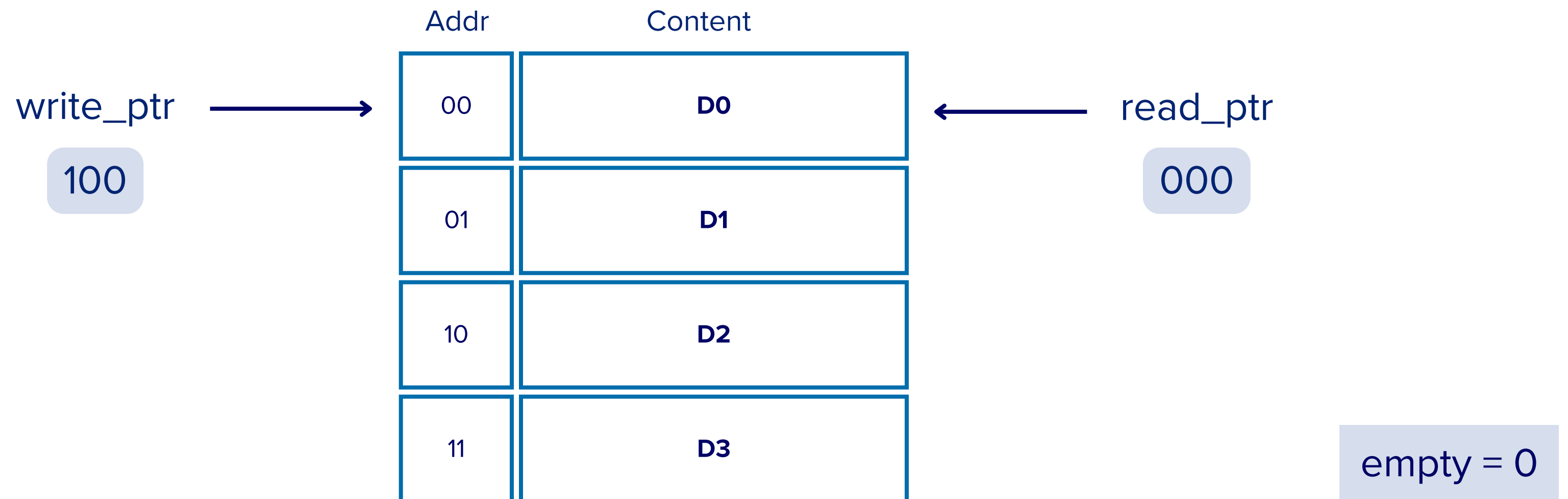
SYNCHRONOUS FIFO DESIGN

- 3rd write



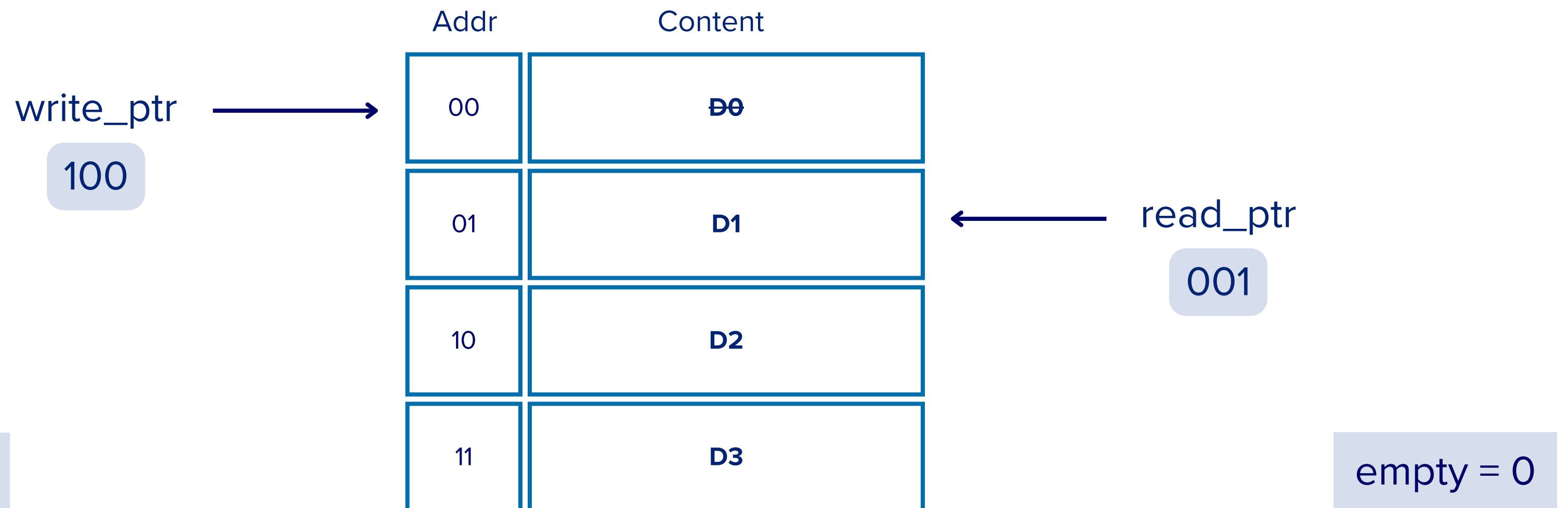
SYNCHRONOUS FIFO DESIGN

- 4th write
- full flag goes high



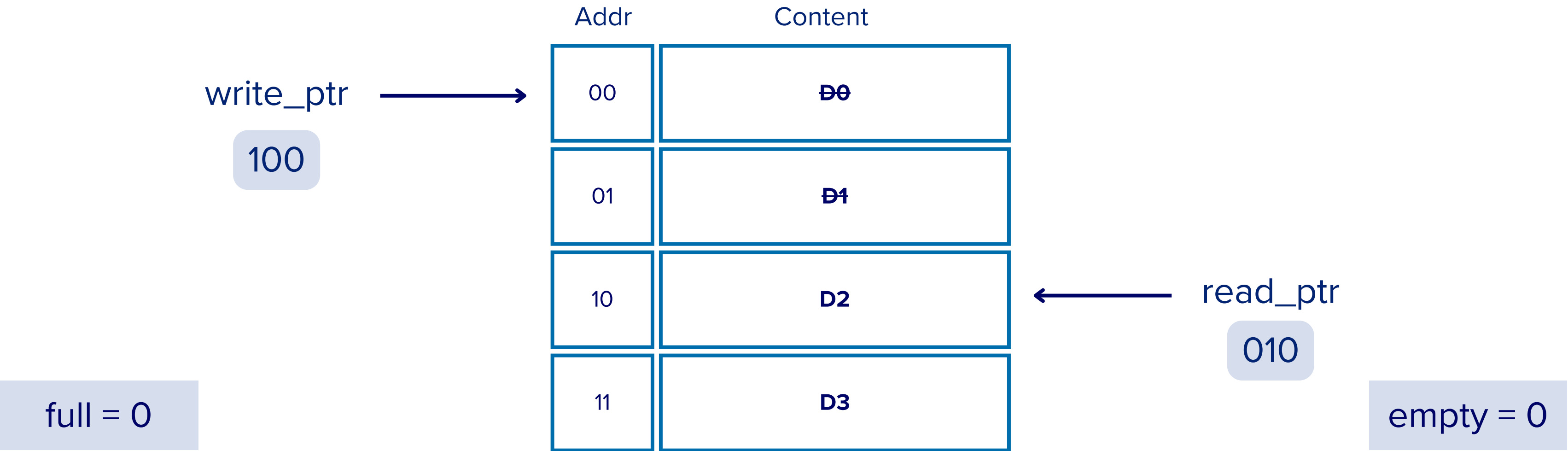
SYNCHRONOUS FIFO DESIGN

- 1st read
- full flag goes low



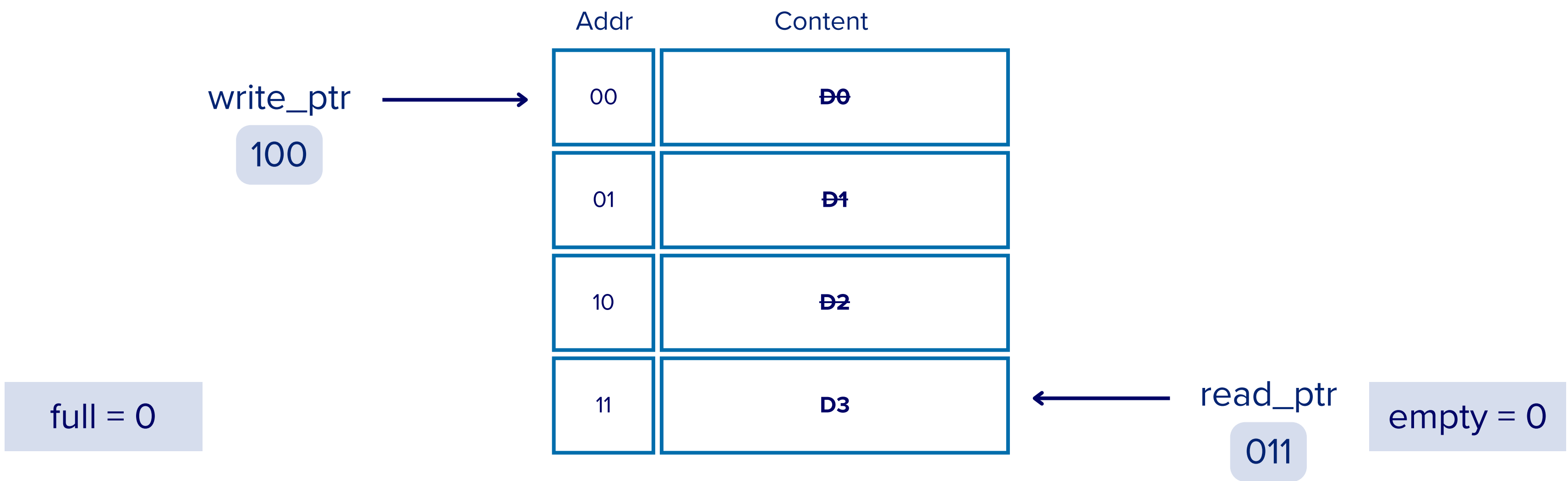
SYNCHRONOUS FIFO DESIGN

- 2nd read



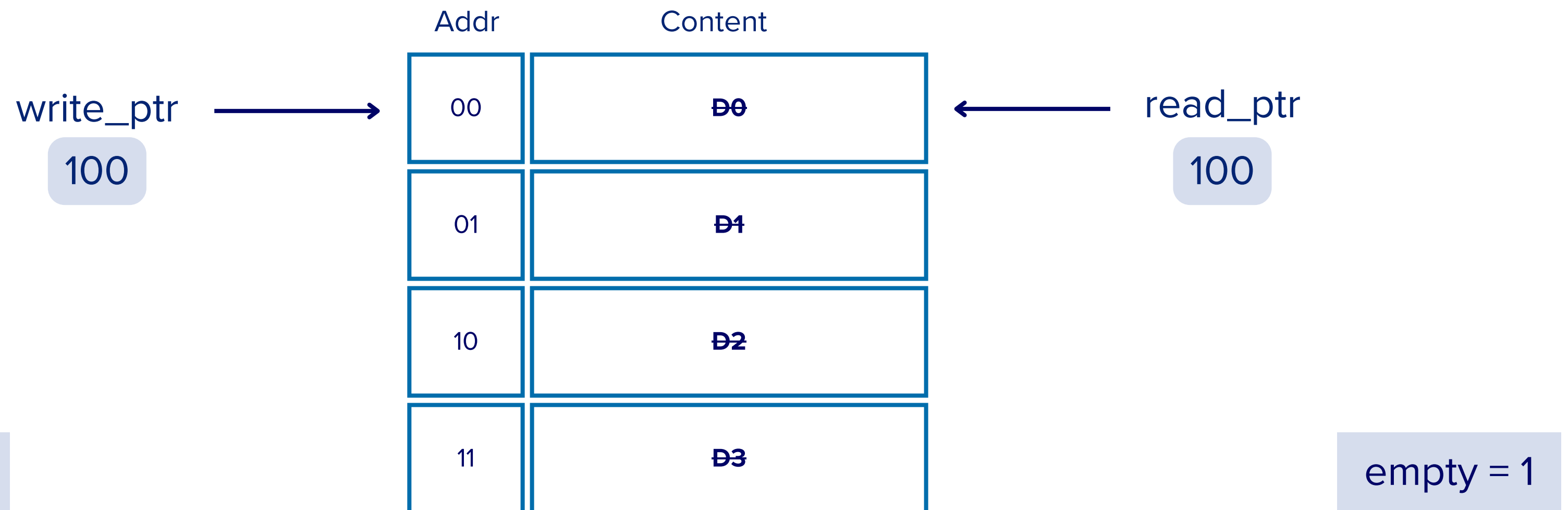
SYNCHRONOUS FIFO DESIGN

- 3rd read

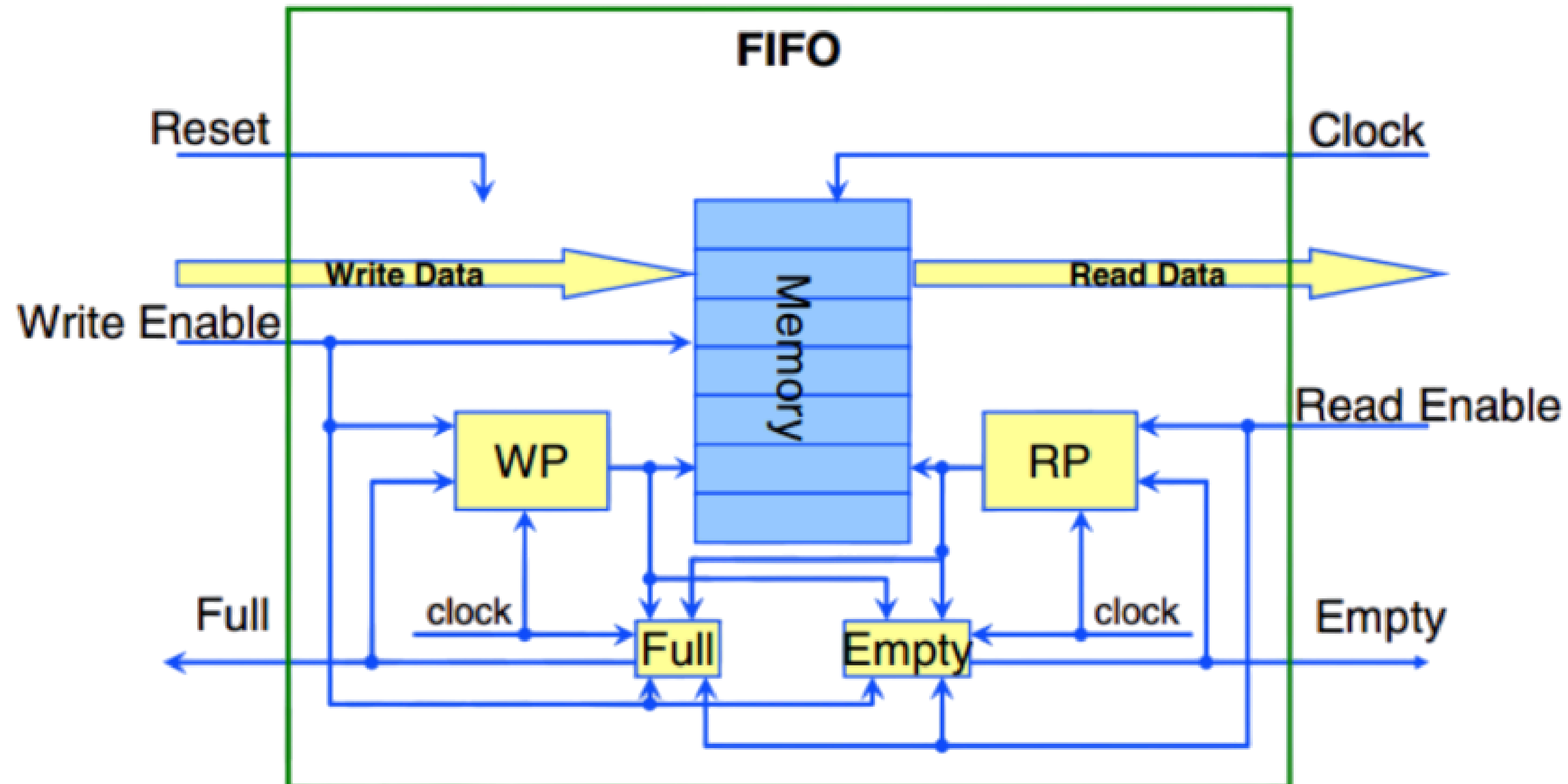


SYNCHRONOUS FIFO DESIGN

- 4th read
- empty flag goes high



SYNCHRONOUS FIFO ARCHITECTURE



The reset goes to WP, RP, full and empty blocks.

SYNCHRONOUS FIFO IMPLEMENTATION

Memory module

```
module fifo_mem #(
    parameter FIFO_WIDTH = 16,
    parameter FIFO_DEPTH = 64,
    parameter ADDR_WIDTH = 6
)((
    input  wire          clk,
    input  wire          w_ena,
    input  wire [FIFO_WIDTH-1:0] w_data,
    input  wire [ADDR_WIDTH-1:0] w_addr,
    input  wire [ADDR_WIDTH-1:0] r_addr,
    output wire [FIFO_WIDTH-1:0] r_data
);

    reg [FIFO_WIDTH-1:0] mem [0:FIFO_DEPTH-1];

    assign r_data = mem[r_addr];

    always @(posedge clk) begin
        if (w_ena) begin
            mem[w_addr] <= w_data;
        end
    end

endmodule
```

SYNCHRONOUS FIFO IMPLEMENTATION

Pointers and flags logic

```
assign wr_addr = wr_ptr[ADDR_WIDTH-1:0];
assign rd_addr = rd_ptr[ADDR_WIDTH-1:0];

// A request is accepted only if the current FIFO state allows it.
assign wr_do = wr_ena && !full;
assign rd_do = rd_ena && !empty;

// Increment the COMPLETE pointers, including the wrap bit.
assign wr_ptr_next =
    wr_ptr + (wr_do ? {{ADDR_WIDTH{1'b0}}, 1'b1}
                  : {(ADDR_WIDTH+1){1'b0}});

assign rd_ptr_next =
    rd_ptr + (rd_do ? {{ADDR_WIDTH{1'b0}}, 1'b1}
                  : {(ADDR_WIDTH+1){1'b0}});

// EMPTY when both next pointers are identical.
assign empty_next = (wr_ptr_next == rd_ptr_next);

// FULL when address portions match but wrap bits differ.
assign full_next =
    (wr_ptr_next[ADDR_WIDTH-1:0] == rd_ptr_next[ADDR_WIDTH-1:0]) &&
    (wr_ptr_next[ADDR_WIDTH] != rd_ptr_next[ADDR_WIDTH]);
```

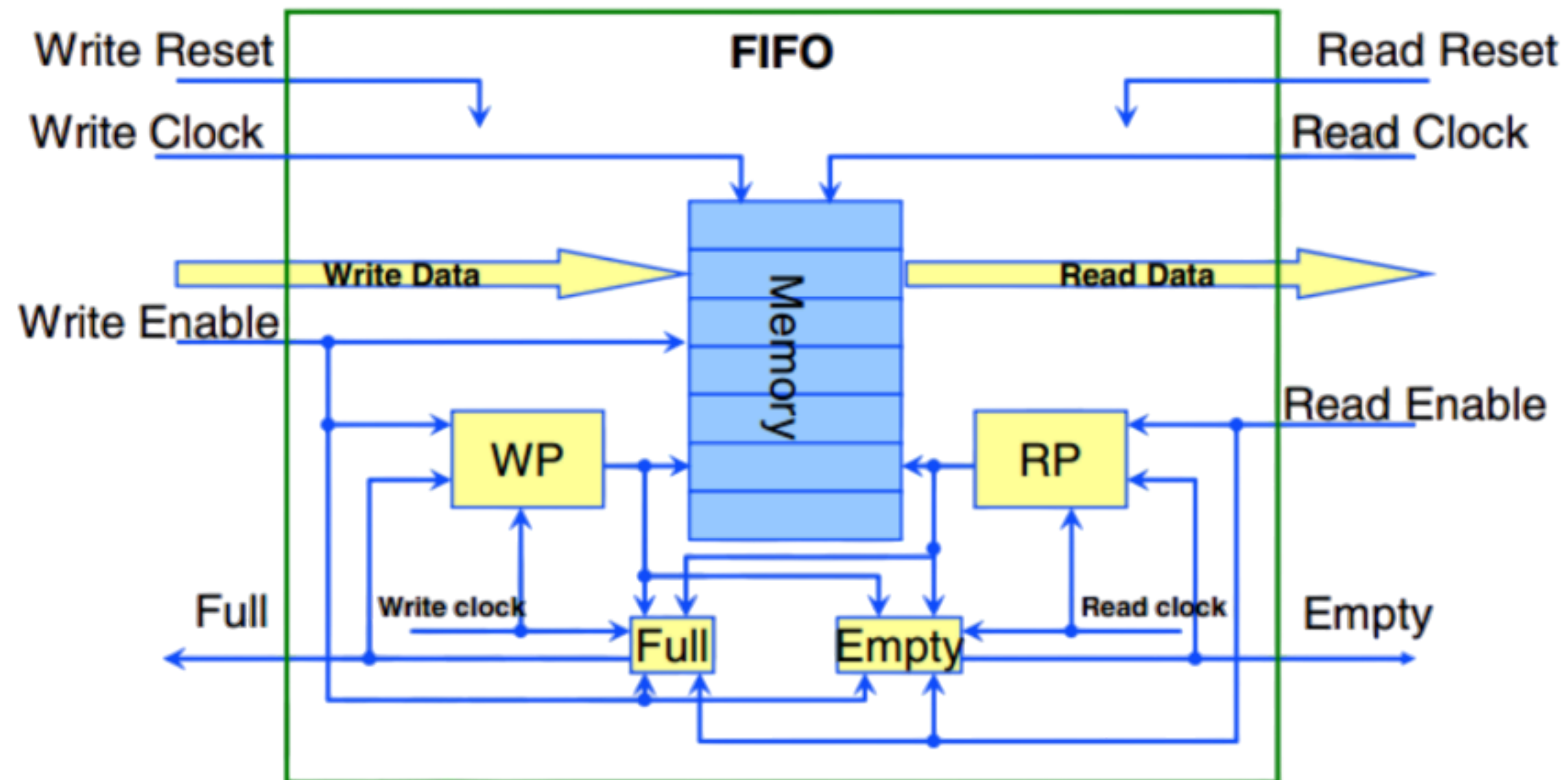
SYNCHRONOUS FIFO IMPLEMENTATION

Pointer and flag registers

```
always @(posedge clk) begin
    if (!rst_n) begin
        wr_ptr <= {(ADDR_WIDTH+1){1'b0}};
        rd_ptr <= {(ADDR_WIDTH+1){1'b0}};
        full   <= 1'b0;
        empty  <= 1'b1;
    end
    else begin
        wr_ptr <= wr_ptr_next;
        rd_ptr <= rd_ptr_next;
        full   <= full_next;
        empty  <= empty_next;
    end
end
```

ASYNCHRONOUS FIFO DESIGN

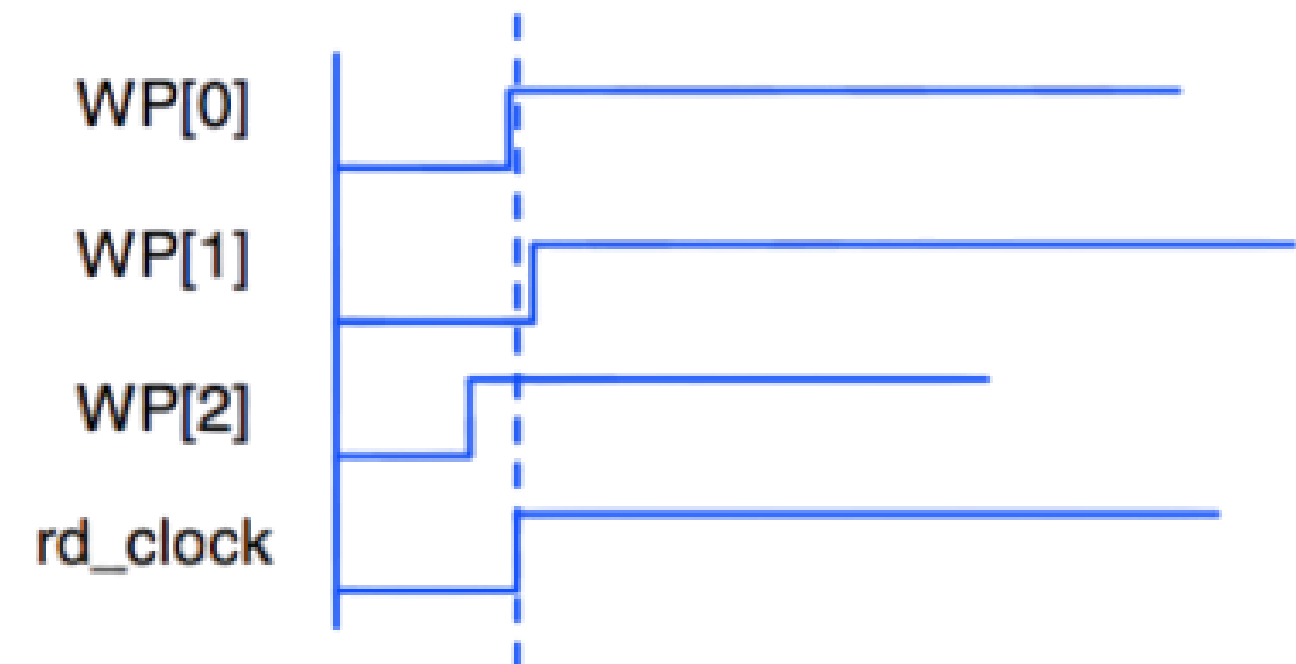
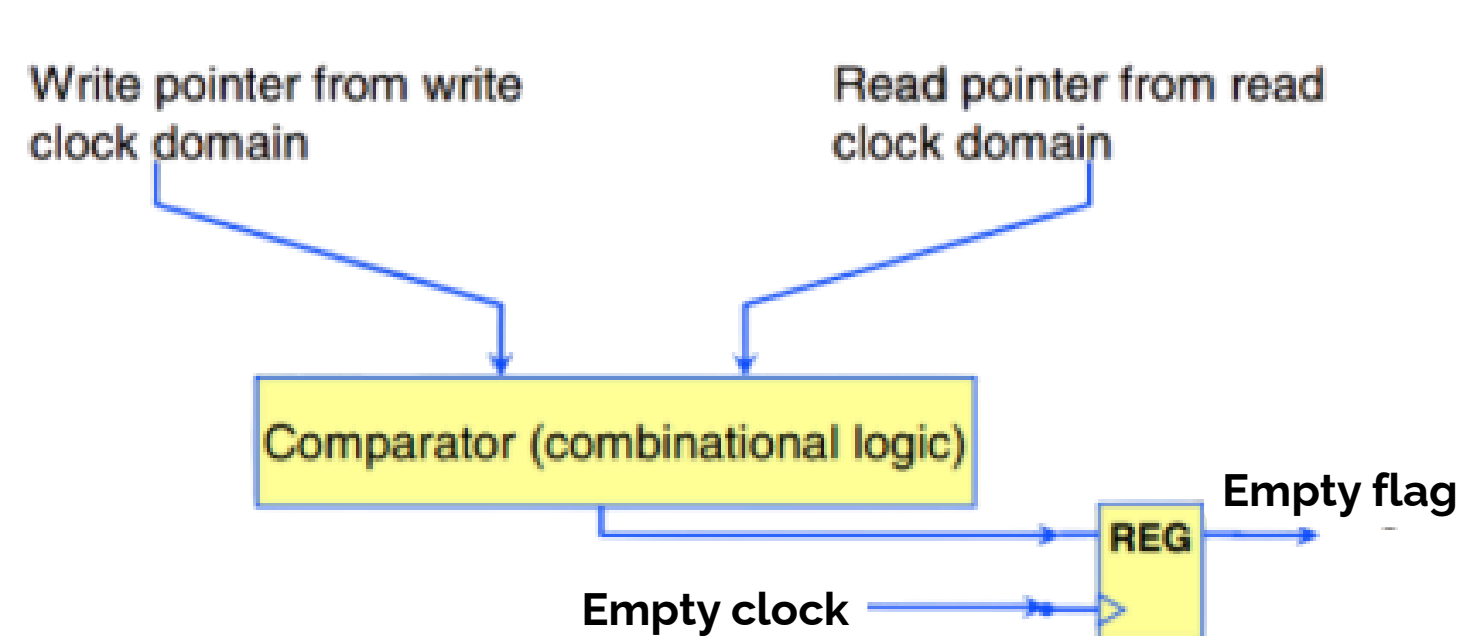
ASYNCHRONOUS FIFO ARCHITECTURE



The write reset goes to WP and full blocks, and read reset goes to RP and empty blocks.

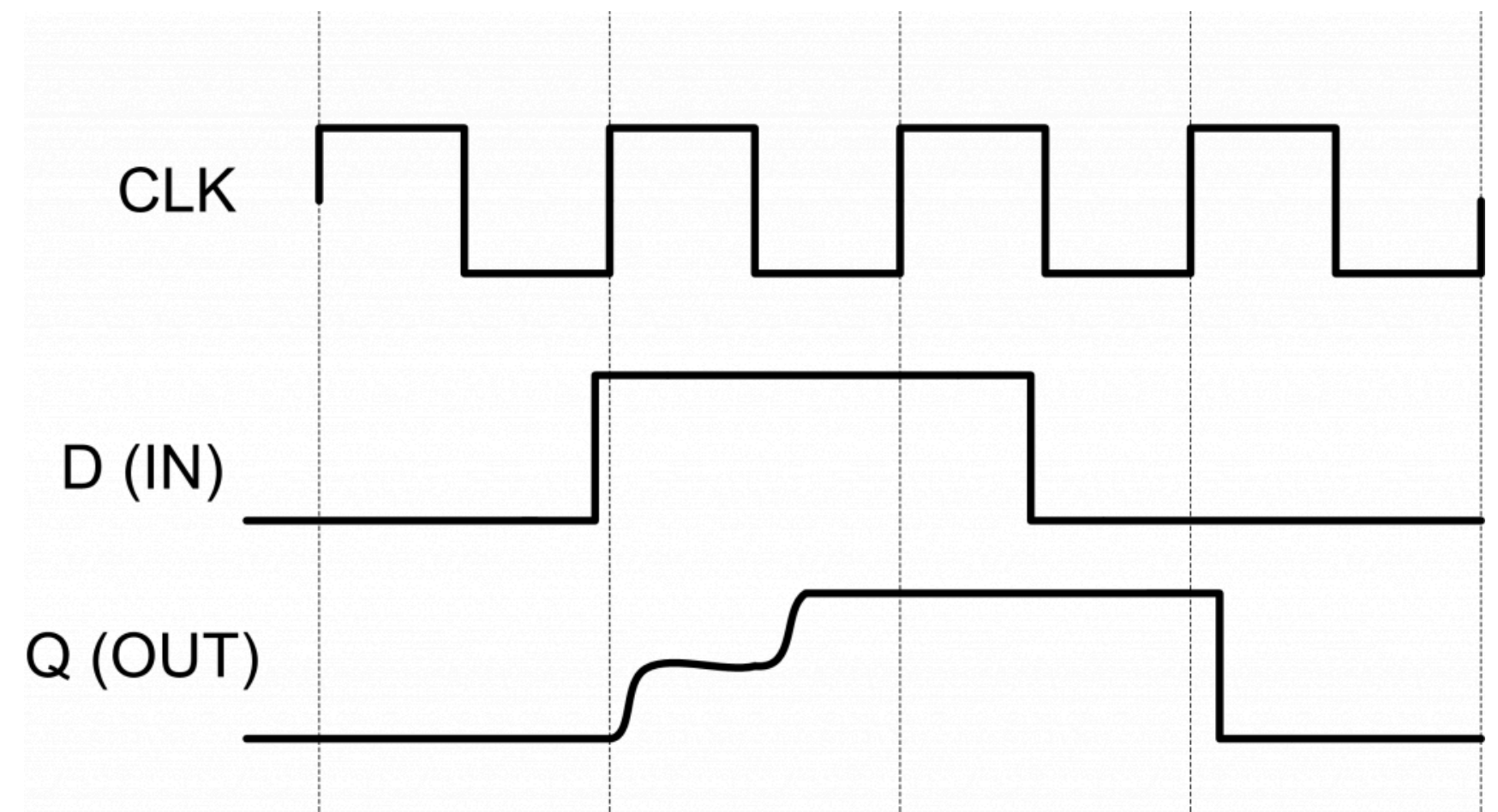
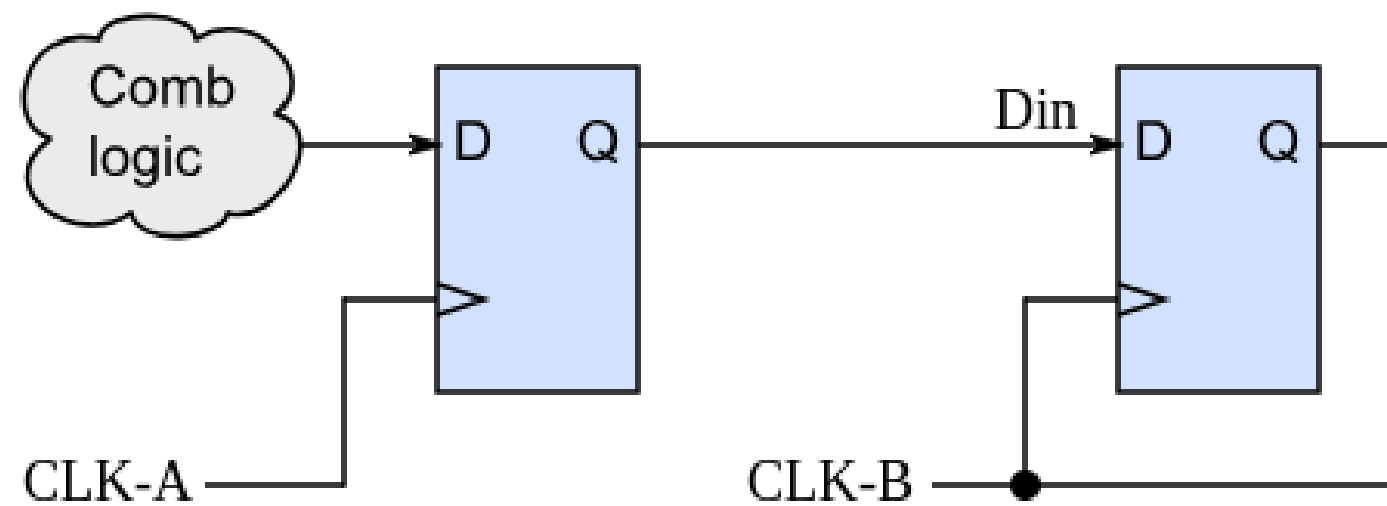
ASYNCHRONOUS FIFO: CHALLENGES

- Glitches may occur due different skew (wrong sampling)
- Clock domain crossing (metastability)



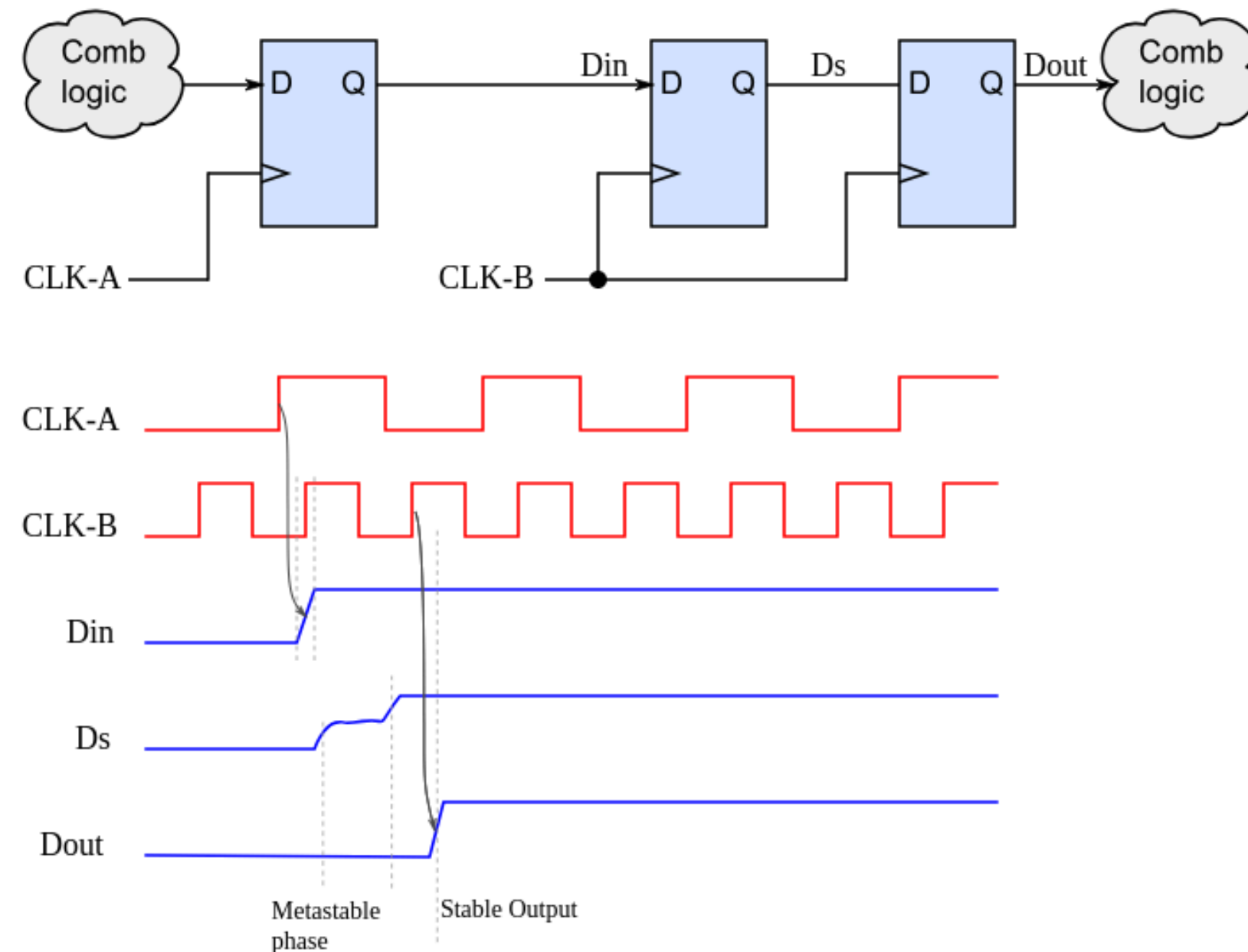
ASYNCHRONOUS FIFO: CHALLENGES

Metastability issue



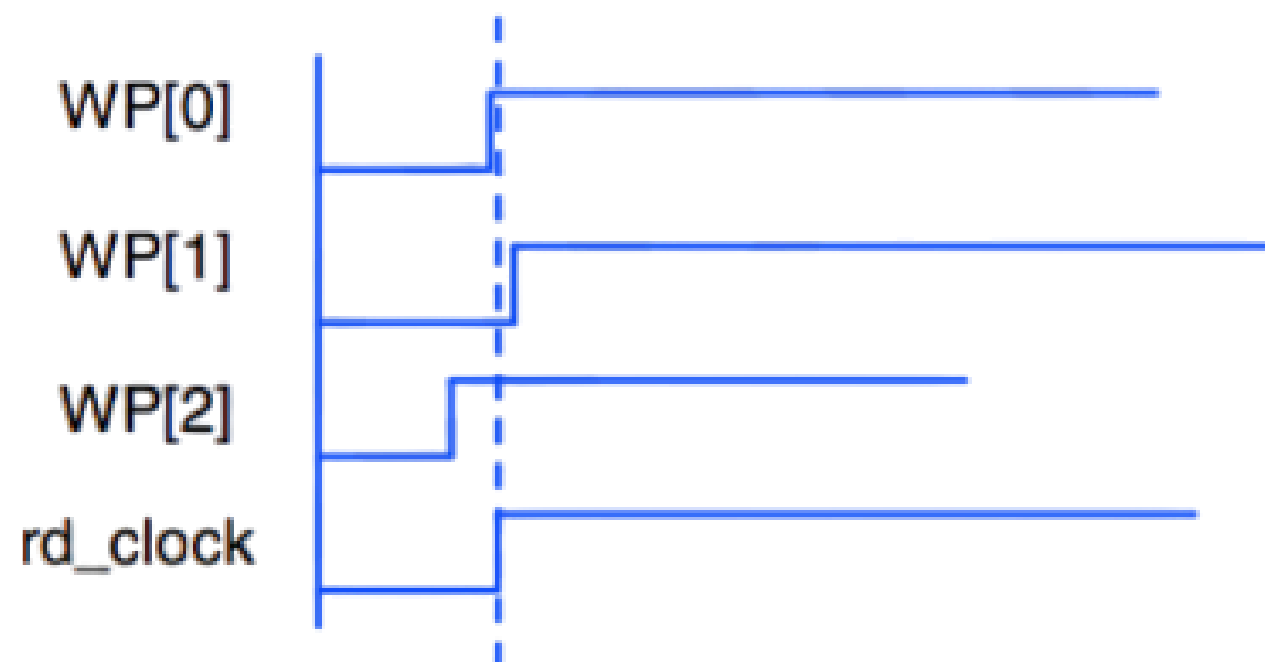
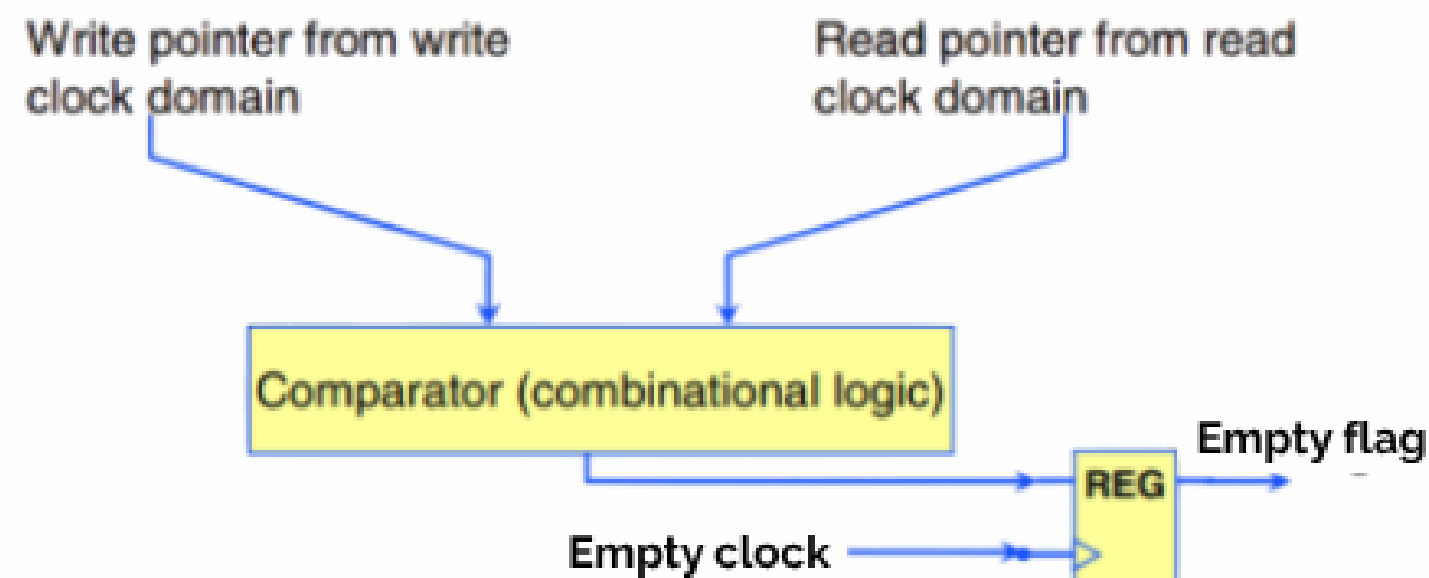
ASYNCHRONOUS FIFO: CHALLENGES

Clock re-synchronization



ASYNCHRONOUS FIFO: CDC

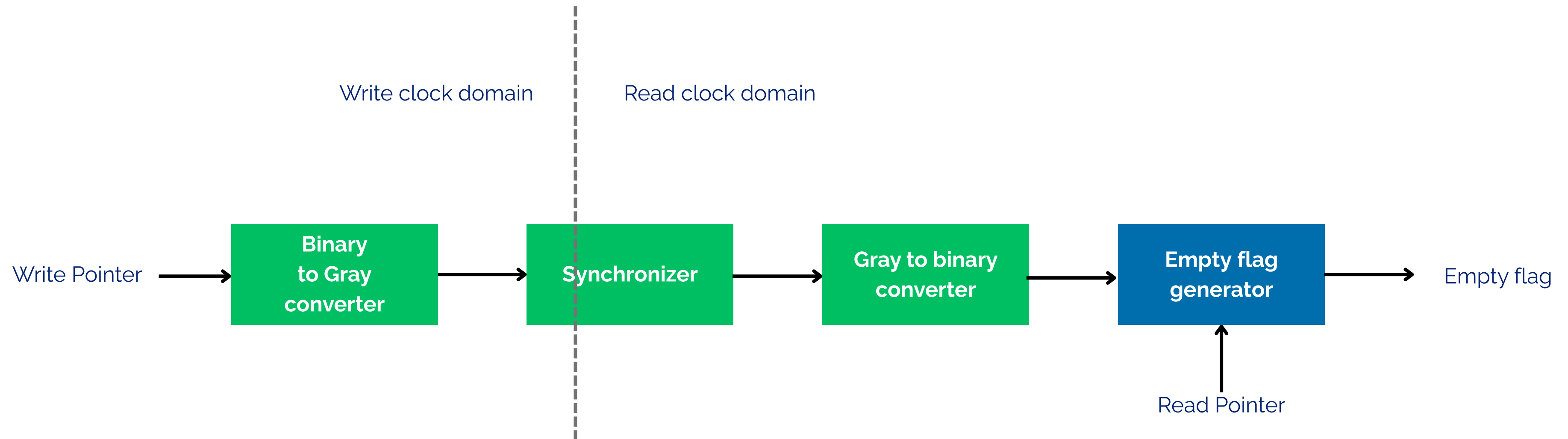
Avoiding glitches in pointers CDC using gray code



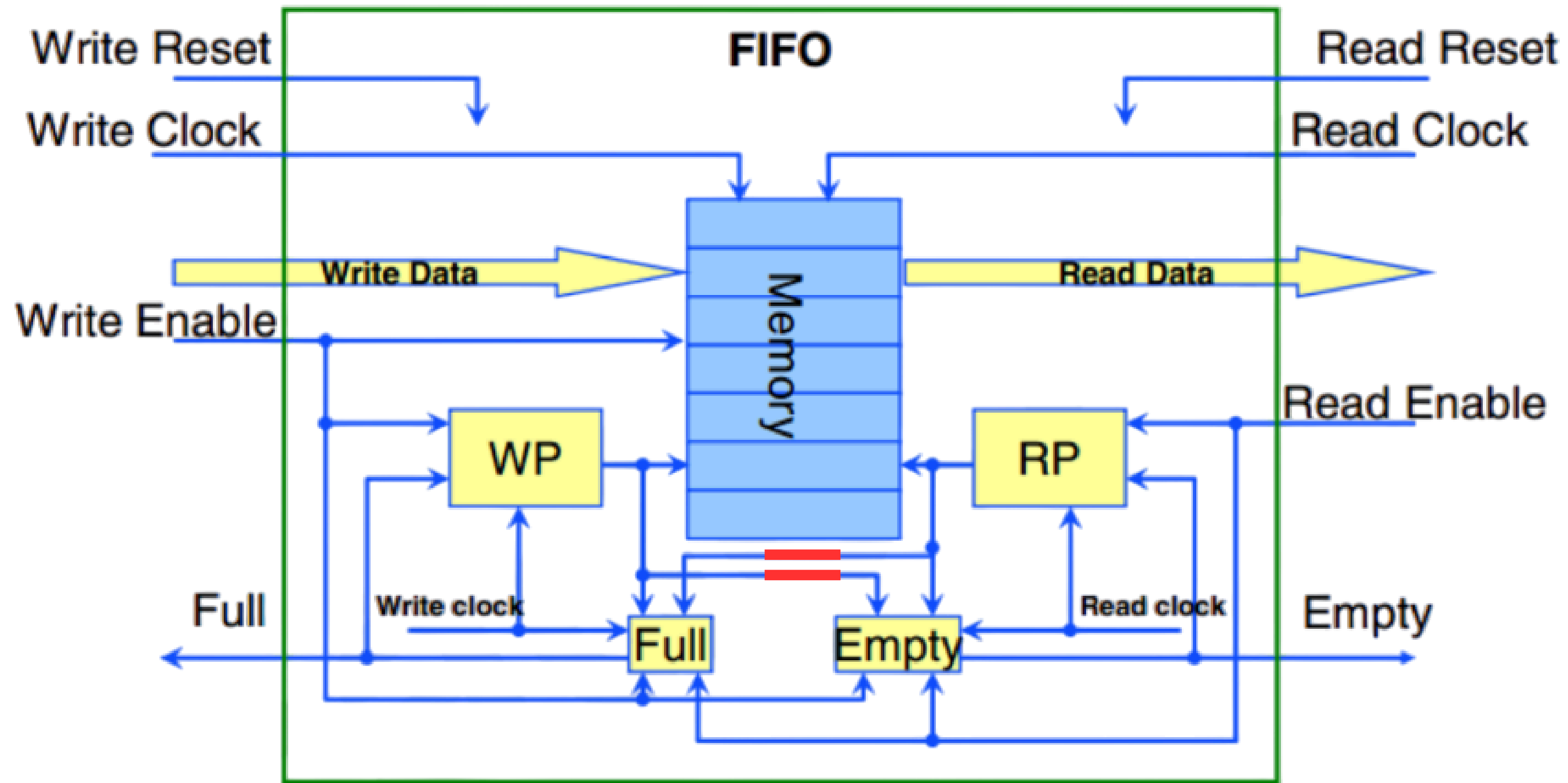
N=3		
Decimal	Binary	Gray
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

ASYNCHRONOUS FIFO

Synchronization solution



ASYNCHRONOUS FIFO: SYNC SOLUTION



The write reset goes to WP and full blocks, and read reset goes to RP and empty blocks.



IGOR D S MIRANDA

igordantas@ufrb.edu.br

[linkedin.com/in/igor-miranda-38117b13](https://www.linkedin.com/in/igor-miranda-38117b13)